Assignment 1 - ACLs, Setuid, And Restricted Sudo

This directory contains C implementations for the assignment commands:

- `fput`
- `fget`
- `create_dir`
- `cd`
- `setacl`
- `getacl`
- `sudo`

The implementation uses per-file and per-directory ACL metadata to mediate access befor
e switching effective UID with `seteuid()` for the requested operation.

Build

Build without privileged setup:

```sh
make clean && make
```

The default `make` target does not run `sudo`, does not change owners, and does not set
 capabilities. This keeps normal compilation usable in non-interactive environments.

Install privileged ownership, setuid bits, and capabilities only inside the assignment
VM:

```sh
sudo make install-privileged
```

That target assumes the VM has a `fakeroot` user and the binaries are intended to be in
stalled as setuid-style coursework tools.

ACL Storage

ACLs are stored in sidecar files:

- File ACL: `.<filename>.acl` in the same directory as the file.
- Directory ACL: `.<dirname>.acl` in the parent directory.

ACL file format:

```text
default:user:rwx
default:group:r-x
default:other:r--
user:<username>:rw-
```

The implementation currently enforces user ACL entries. Group and other entries are sto
red as metadata defaults but are not a full Linux ACL replacement.

Command Usage

`fput`

```sh
./fput FILENAME "text"
```

Creates or appends to `FILENAME`. For a new file, the parent directory ACL is checked w
hen present. If no parent ACL exists, the implementation falls back to ordinary DAC beh
avior so first-time creation in normal directories remains usable. A new ACL sidecar is
 created from the file mode.

`fget`

```sh
./fget FILENAME
```

Prints file content when the invoking user has read permission in the file ACL.

`create_dir`

```sh
./create_dir [-p|--parents] [-v|--verbose] DIRECTORY...
```

Creates directories after checking write and execute permissions on relevant parent ACL
s when present. `-p` creates intermediate directories.

`cd`

```sh
./cd DIRECTORY
```

Checks execute permission on the target path and calls `chdir()`. Because a child proce
ss cannot change the parent shell's working directory, this behaves as a permission-che
cking demonstration command rather than a persistent shell builtin.

`getacl`

```sh
./getacl FILE_OR_DIRECTORY
```

Prints the ACL sidecar when the caller has read permission.

`setacl`

```sh
./setacl -m u:<username>:<rwx> FILE_OR_DIRECTORY
./setacl -x u:<username>:--- FILE_OR_DIRECTORY
```

Adds, modifies, or clears a user ACL entry. The target user must exist in the system us
er database.

`sudo`

```sh
./sudo <fput|fget|create_dir|cd|setacl|getacl> [args...]
```

Runs only the allowed assignment commands from `/fakeroot`. The wrapper rejects path tr
aversal and shell metacharacter execution by using `fork()` plus `execv()` directly. It
 sets `SUDO_MODE=1` so the called command checks execute permission on the command bina
ry ACL before applying the requested operation.

Hardening Notes

- Normal compilation no longer requires interactive `sudo`.
- The custom `sudo` wrapper no longer uses `system()`.
- ACL filename construction is bounded.
- Malformed ACL lines fail closed.
- File read/write loops handle partial reads/writes and `EINTR`.
- ACL updates are written through a temporary file and renamed.

Verification

```sh
make clean && make
tmp=$(mktemp -d)
./fput "$tmp/sample.txt" hello
./fget "$tmp/sample.txt"
rm -rf "$tmp"
```

Privileged behavior must be tested inside the assignment VM after `sudo make install-privileged`.

Assumptions

- The assignment VM contains real Linux users for ACL entries.
- Privileged testing uses a `fakeroot` owner as described in the assignment.
- ACL sidecar files are trusted metadata for this coursework implementation; this is not a production MAC or Linux ACL implementation.